

FIGURA 2.4. Encadenar peticions (II): Fetch i Promises

Què hem fet aquí? Doncs una primera petició per trobar la informació de la pel·lícula:

```
1 const response = await fetch(apiUrl);
```

I ho hem fet molt còmodament perquè sabem que ens esperarem que les dades ens arribin gràcies a la partícula `await`.

Després recorrem tot l'array de personatges fent:

```
1 data.people.forEach(url => {  
2   personatgesPromises.push(fetch(url).then(res => res.json()))  
3   });
```

Però clar, quan acabaran les peticions? No ho sabem, i quasi que no ens importa, perquè fent...

```
1 const personatgesData = await Promise.all(personatgesPromises);
```

..., sabrem que ja hauran acabat i automàticament tindrem les dades que volem a la variable `personatgesData`.

3. Projecte amb AJAX i bases de dades

Un cop entesos els conceptes de la comunicació asíncrona, és el moment de posar-los en pràctica mitjançant dos projectes reals que simulen entorns de treball professionals. En aquests exemples, veurem com el JavaScript del navegador (*frontend*) interactua amb un servidor i una base de dades, o amb serveis externs de tercers, per crear experiències d'usuari dinàmiques i eficients.

En primer lloc, desenvoluparem una aplicació de gestió de tasques des de zero, on controlarem tant el client com el servidor. En segon lloc, explorarem com consumir dades d'una API externa (**Open Library**) per crear un cercador de llibres avançat amb paginació i detalls en temps real.

3.1 Projecte CRUD

Aprofitant les noves competències adquirides en l'àmbit de la comunicació asíncrona, concretament en l'ús d'AJAX, voldrem crear una aplicació de llista de tasques, on ens encarregarem de gestionar el *frontend* i delegarem la gestió de la base de dades en un *backend*.

Perquè aquest *frontend* i aquest *backend* es comuniquin ho farem mitjançant peticions asíncrones.

El backend, l'escriurem amb **nodeJS** tal com vàrem veure en la lliçó 5 Framework Express. Serà una aplicació semblant a la que vàrem crear en aquella lliçó, però ara amb una voluntat de crear quelcom més elaborat i realista i on es faci evident i útil l'ús de la comunicació asíncrona.

El frontend l'escriurem als arxius `app.js` i `index.html`, i complirà els següents requisits:

- S'haurà de poder crear una tasca nova.
- S'haurà de poder modificar el text d'una tasca.
- S'haurà de poder marcar una tasca com a completada.
- S'haurà de poder eliminar una tasca.

A més a més, la nostra aplicació ha de ser capaç d'avisar a l'usuari si perd la connexió amb l'*endpoint*, i així evitar que saltin errors en el moment de realitzar les accions, en cas de què aquestes no es poden dur a terme. I un cop es torni a restablir la connexió el comportament de l'aplicació ha de ser l'habitual.

3.1.1 Estructura del projecte

Per començar amb el projecte, el primer pas serà configurar l'entorn de treball. Crearem una nova carpeta per al nostre projecte i inicialitzarem un projecte Node.js utilitzant npm. A continuació, instal·larem **Express**, que serà el **framework** que utilitzarem per gestionar les rutes i les peticions HTTP. També instal·larem **cors** perquè el **backend** accepti peticions des de qualsevol origen. I finalment instal·larem **sqlite3** per tenir la persistència de les tasques.

```
1 mkdir crud
2 cd crud
3 npm init -y
4 npm install express cors sqlite3
```

A continuació crearem els següents arxius:

- `index.html`: que contindrà el **layout** del projecte i serà el punt d'entrada de l'aplicació.
- `styles.css`: amb els estils gràfics.
- `app.js`: serà la lògica del **frontend**.
- `server.js`: serà la lògica del **backend**.
- `tasks.db`: serà la base de dades. Aquest arxiu no caldrà crear-lo manualment ja que es crearà automàticament quan s'aixequi el servei

3.1.2 Layout del projecte (index.html)

Per enriquir l'aplicació utilitzarem el framework CSS **Bootstrap**, la llibreria **jQuery** i les llibreries **SweetAlert** i **Toastr.js** per mostrar missatges.

L'estructura de l'aplicació serà molt simple: un petit camp d'entrada de text amb el seu botó per afegir la tasca i un llistat de tasques que s'anirà carregant poc a poc.

El codi podria ser:

```
1 <!DOCTYPE html>
2 <html lang="ca">
3
4 <head>
5   <meta charset="UTF-8">
6   <meta name="viewport" content="width=device-width, initial-
   scale=1.0">
7   <title>Gestió de Tasques</title>
8
9   <!-- jQuery -->
10  <script src="https://code.jquery.com/jquery-3.7.1.min.js">
```

```
11 integrity="sha256-/JqT3SQfawRcv/BIHPThkBs00EvtFFmqPF/  
    lYI/Cxo=" crossorigin="anonymous"></script>  
12 <!-- Bootstrap --> <link href="https://cdn.jsdelivr.net/npm/  
    bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">  
13 <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/  
    dist/js/bootstrap.bundle.min.js"></script>  
14  
15 <!-- SweetAlert2 -->  
16 <script src="https://cdn.jsdelivr.net/npm/sweetalert2@11"></  
    script>  
17  
18 <!-- Font Awesome -->  
19 <link rel="stylesheet" href="https://cdnjs.cloudflare.com/  
    ajax/libs/font-awesome/6.4.0/css/all.min.css">  
20  
21 <!-- Toastr.js -->  
22 <link href="https://cdnjs.cloudflare.com/ajax/libs/toastr.js  
    /latest/css/toastr.min.css" rel="stylesheet" />  
23 <script src="https://cdnjs.cloudflare.com/ajax/libs/toastr.  
    js/latest/js/toastr.min.js"></script>  
24  
25 <!-- Codi del programa -->  
26 <link rel="stylesheet" href="styles.css">  
27 <script src="app.js"></script>  
28 </head>  
29  
30 <body class="bg-light">  
31 <div class="container d-flex justify-content-center align-  
    items-center min-vh-100">  
32 <div class="card p-4 d-flex flex-column">  
33 <h3 class="text-center mb-3 text-primary">Llista de  
    Tasques</h3>  
34 <pre class="text-center mb-3" id="connection">  
    Connexió: <i class="fa-solid fa-thumbs-up"></i></  
    pre>  
35 <div class="input-group mb-3">  
36 <input type="text" id="taskInput" class="form-  
    control" placeholder="Afegeix una tasca">  
37 <button class="btn btn-primary" onclick="addTask  
    ()"><i class="fas fa-plus"></i></button>  
38 </div>  
39 <div class="list-container">  
40 <ul class="list-group" id="taskList"></ul>  
41 </div>  
42 </div>  
43 </div>  
44  
45 <script>  
46  
47 </script>  
48 </body>  
49  
50 </html>
```

3.1.3 Estils gràfics (styles.css)

Per completar els estils de Bootstrap crearem algunes classes personalitzades:

```
1 body { background: linear-gradient(180deg, #654ba2, #667eea);  
2 }  
3  
4 .card {  
5     border-radius: 15px;  
6     background: white;  
7     box-shadow: 0 10px 20px rgba(0, 0, 0, 0.2);  
8     width: 100%;  
9     max-width: 500px;  
10 }  
11  
12 .list-container {  
13     max-height: 300px;  
14     overflow-y: auto;  
15 }  
16  
17 .list-group-item {  
18     border-radius: 8px;  
19     transition: all 0.3s;  
20 }  
21  
22 .list-group-item:hover {  
23     background: rgba(0, 0, 0, 0.05);  
24 }  
25  
26 .btn {  
27     border-radius: 50px;  
28     padding: 8px 15px;  
29     font-weight: bold;  
30     transition: transform 0.2s, box-shadow 0.2s;  
31 }  
32  
33 .btn:hover {  
34     transform: scale(1.1);  
35     box-shadow: 0 4px 10px rgba(0, 0, 0, 0.2);  
36 }  
37  
38 .btn i {  
39     color: white;  
40 }  
41  
42 .list-container::-webkit-scrollbar {  
43     width: 8px;  
44 }  
45  
46 .list-container::-webkit-scrollbar-track {  
47     background: #f0f0f0;  
48     border-radius: 10px;  
49 }  
50  
51 .list-container::-webkit-scrollbar-thumb {  
52     background: #764ba2;  
53     border-radius: 10px;  
54 }  
55  
56 .list-container::-webkit-scrollbar-thumb:hover {  
57     background: #5a3b87;  
58 }  
59  
60 .completed { text-decoration: line-through; color: gray; }
```

3.1.4 Lògica del frontend (app.js)

El que ha de fer el *frontend* és fer peticions a l'endpoint creat per `server.js` per tal d'enviar-li tasques, llistar tasques, editar tasques i eliminar tasques, en essència, la feina d'un **CRUD** (*Create*, *Read*, *Update*, *Delete*).

```
1  const API_URL = "http://localhost:3000/tasks";
2  let taskList = null;
3  let taskInput = null;
4  let connection = true;
5  let showedErrorMessage = false;
6  let showedConnectionMessage = false;
7  let refreshTime = 5000;
8  let connectionTime = 500;
9
10 /** Carregar tasques en iniciar */
11 window.onload = function () {
12     init();
13
14     loadTasks();
15
16     setInterval(() => {
17         checkConnection();
18     }, connectionTime);
19 };
20
21 /** Funció per agafar aquells elements del DOM que necessitem */
22 function init() {
23     taskList = document.getElementById("taskList");
24     taskInput = document.getElementById("taskInput");
25 }
26
27 /** Funció per carregar les tasques des de l'endpoint */
28 function loadTasks() {
29     if (!connection) {
30         return;
31     }
32
33     let xhr = new XMLHttpRequest();
34     xhr.open("GET", API_URL, true);
35     xhr.onreadystatechange = function () {
36         if (xhr.readyState === 4) {
37             if (xhr.status === 200) {
38                 let tasks = JSON.parse(xhr.responseText);
39
40                 renderTasks(tasks);
41             }
42         }
43     };
44     xhr.send();
45 }
46
47 /** Petició al endpoint per afegir una tasca nova. */
48 function addTask() {
49     let text = taskInput.value.trim();
50     if (text === "") return Swal.fire({
51         title: "Error",
52         text: "El text de la tasca no pot estar buit.",
53         icon: "error",
```

```
54     });
55     if (!connection) {         Swal.fire({
56         title: "Error",
57         text: "No es pot afegir una tasca sense connexió. Si
            us plau, torna a intentar-ho quan la connexió s'
            hagi restablert.",
58         icon: "error",
59     });
60
61     return;
62 }
63
64 let xhr = new XMLHttpRequest();
65 xhr.open("POST", API_URL, true);
66 xhr.setRequestHeader("Content-Type", "application/json");
67
68 xhr.onreadystatechange = function () {
69     if (xhr.readyState === 4 && xhr.status === 200) {
70         taskInput.value = "";
71         loadTasks();
72     }
73 };
74
75 xhr.send(JSON.stringify({ text }));
76 }
77
78 /** Petició al endpoint per actualitzar una tasca (completada o
    text). */
79 function updateTask(id, data) {
80     if (!connection) {
81         alert("No es pot actualitzar una tasca sense connexió.
            Si us plau, torna a intentar-ho quan la connexió s'
            hagi restablert.");
82
83         return;
84     }
85
86     let xhr = new XMLHttpRequest();
87     xhr.open("PUT", `${API_URL}/${id}`, true);
88     xhr.setRequestHeader("Content-Type", "application/json");
89
90     xhr.onreadystatechange = function () {
91         if (xhr.readyState === 4 && xhr.status === 200) {
92             loadTasks();
93         }
94     };
95
96     xhr.send(JSON.stringify(data));
97 }
98
99 /** Petició al endpoint per eliminar una tasca. */
100 function deleteTask(id) {
101     if (!connection) {
102         alert("No es pot eliminar una tasca sense connexió. Si
            us plau, torna a intentar-ho quan la connexió s'hagi
            restablert.");
103
104         return;
105     }
106
107
108     let xhr = new XMLHttpRequest();
109     xhr.open("DELETE", `${API_URL}/${id}`, true);
```

```
110
111 xhr.onreadystatechange = function () {          if (xhr.readyState
        === 4 && xhr.status === 200) {
112     loadTasks();
113     }
114 };
115
116 xhr.send();
117 }
118
119 /** Funció auxiliar per marcar una tasca com a completada */
120 function toggleComplete(id, text, completed) {
121     completed = completed == 0 ? 1 : 0;
122     updateTask(id, { text: text, completed: completed });
123 }
124
125 /** Funció auxiliar per editar el text d'una tasca */
126 function editTask(id, newText, completed) {
127     if (newText !== null && newText.trim() !== "") {
128         updateTask(id, { text: newText, completed: completed });
129     }
130 }
131
132
133 /** Funció que cada pocs segons fa una petició a l'API per saber
    si tenim connexió o no. */
134 function checkConnection() {
135     let xhr = new XMLHttpRequest();
136     xhr.open("GET", `${API_URL}`, true);
137
138     xhr.onreadystatechange = function () {
139         if (xhr.readyState === 4) {
140             if (xhr.status === 200) {
141                 connection = true;
142             } else {
143                 connection = false;
144             }
145         }
146     };
147
148     xhr.send();
149
150     showConnectionMessage();
151 }
152
153 /** Funció que fa sortir missatges de l'estat de la connexió grà
    cies a la llibreria toastr */
154 function showConnectionMessage() {
155     if (!connection && !showedErrorMessage) {
156         toastr.error("S'ha perdut la connexió.", "Error de
            connexió");
157         showedErrorMessage = true;
158         showedConnectionMessage = false;
159         document.getElementById("connection").innerHTML = "
            Connexió: <i class='fa-solid fa-thumbs-down'></i>";
160     }
161
162     if (connection && !showedConnectionMessage) {
163         toastr.success("Connexió correcta amb l'endpoint.", "
            Connexió establerta", { timeOut: 5000 });
164         showedConnectionMessage = true;
165         showedErrorMessage = false;
166         document.getElementById("connection").innerHTML = "
```



```

    Connexió: <i class='fa-solid fa-thumbs-up'></i>;
167 }}
168
169 /** Funció per renderitzar les tasques a la interfície */
170 function renderTasks(tasks) {
171     taskList.innerHTML = "";
172     tasks.forEach(task => {
173         let li = document.createElement('li');
174         li.className = 'list-group-item d-flex justify-content-
            between align-items-center p-3 shadow-sm';
175
176         let span = document.createElement('span');
177         span.textContent = `${task.text}`;
178         span.className = task.completed == 0 ? 'fw-bold' : 'fw-
            bold completed';
179
180         let div = document.createElement('div');
181
182         let completeBtn = document.createElement('button');
183         completeBtn.className = 'btn btn-success btn-sm me-2
            rounded-pill';
184         completeBtn.innerHTML = '<i class="fas fa-check"></i>';
185         completeBtn.onclick = function () {
186             toggleComplete(`${task.id}`, `${task.text}`, `${task
                .completed}`);
187         };
188
189         let editBtn = document.createElement('button');
190         editBtn.className = 'btn btn-warning btn-sm me-2 rounded
            -pill';
191         editBtn.innerHTML = '<i class="fas fa-edit"></i>';
192         editBtn.onclick = function () {
193             Swal.fire({
194                 title: 'Edita la tasca',
195                 input: 'text',
196                 inputValue: span.textContent,
197                 showCancelButton: true,
198                 confirmButtonText: 'Guardar',
199                 cancelButtonText: 'Cancel·lar',
200             }).then((result) => {
201                 if (result.isConfirmed && result.value.trim()
                    !== '') {
202                     editTask(`${task.id}`, result.value, `${task
                        .completed}`);
203                 }
204             });
205         };
206
207         let deleteBtn = document.createElement('button');
208         deleteBtn.className = 'btn btn-danger btn-sm rounded-
            pill';
209         deleteBtn.innerHTML = '<i class="fas fa-trash"></i>';
210         deleteBtn.onclick = function () {
211             deleteTask(`${task.id}`);
212         };
213
214         div.appendChild(completeBtn);
215         div.appendChild(editBtn);
216         div.appendChild(deleteBtn);
217
218         li.appendChild(span);
219         li.appendChild(div);
220     });
}
```

```
221 document.getElementById('taskList').appendChild(li);  
222 });}
```

El codi anterior gestiona tota la interacció de l'usuari i la comunicació asíncrona amb el servidor. La seva estructura es divideix en quatre pilars fonamentals:

1. **Gestió de l'Estat i Connexió:** l'aplicació monitoritza constantment si el servidor està actiu mitjançant la funció `checkConnection()`. Si es perd la connexió, el sistema desactiva les funcionalitats d'edició i avisa l'usuari amb notificacions visuals gràcies a la llibreria **Toastr**.
2. **Operacions CRUD Asíncrones:** s'utilitza l'objecte `XMLHttpRequest` per realitzar peticions HTTP al backend:
 - **GET (loadTasks):** recupera la llista de tasques i la passa a la funció de renderitzat.
 - **POST (addTask):** envia una nova tasca al servidor després de validar que el text no estigui buit.
 - **PUT (updateTask):** permet modificar tant el text d'una tasca com el seu estat (completada o pendent).
 - **DELETE (deleteTask):** sol·licita l'eliminació permanent d'una tasca a la base de dades.
3. **Renderització Dinàmica del DOM:** La funció `renderTasks()` s'encarrega de "dibuixar" la interfície. Per cada tasca rebuda, crea elements HTML (`<li>`, `<span>`, `<button>`) i els injecta al contenidor principal, assignant-los els estils de Bootstrap i els esdeveniments de clic corresponents.
4. **Interfície d'Usuari Avançada:** per millorar l'experiència (UX), s'integren llibreries externes com **SweetAlert2** per mostrar diàlegs de confirmació i formularis d'edició més elegants que els prompt natius del navegador.

3.1.5 La lògica del backend (server.js)

La primera missió del backend és aixecar un servidor que mantindrà actiu l'endpoint que anomenarem, tal com deduïm del codi del frontend, `tasks` (<http://localhost:3000/tasks>)

```
1 const express = require("express");  
2 const sqlite3 = require("sqlite3").verbose();  
3 const cors = require("cors");  
4  
5 const app = express();  
6 const PORT = 3000;  
7  
8 // Middleware per permetre peticions JSON i CORS  
9 app.use(express.json());  
10 app.use(cors());  
11
```